

# Tide retrospective:

## *2D cleanup — refactor through AddSeparable*

Daniel Murfet (with Claude Opus 4.7)

6 May 2026

### Abstract

A cleanup Tide chained on top of Tide 12 (separable-potential abstraction). The two existing 2D files (`SemiDegenerate.lean`, `PureQuartic.lean`) are refactored to apply the new `AddSeparable` lemmas via small `[simp]`-tagged bridge lemmas, instead of re-deriving `integral_prod_mul` plumbing case-by-case. Net delta is  $-22$  lines across three files (the abstraction itself shed an unused hypothesis package). Public theorem statements unchanged on the headline `cov_affine_*` and moment lemmas; only internal proof structure changes. Build green on first compile, zero `sorry/axiom/native_decide`. The interesting process moves were two: a deliberate API regression on `AddSeparable` (drop hypotheses that Tide 12 added as discipline; the concrete refactor demonstrated they blocked rather than helped downstream callers), and a worktree-isolated tide run that confirmed the structural fix to the recurring concurrent-agent branch race.

## 1 Setting

This is a cleanup Tide rather than a step outward. Tide 12 landed the `Laplace.TwoD.AddSeparable` module, which exposes for any  $L(x, y) = U(x) + V(y)$  the generic factorisations  $Z_{2D}(t) = Z_U(t) \cdot Z_V(t)$  and  $\langle f \otimes g \rangle = \langle f \rangle_U \cdot \langle g \rangle_V$ . Two existing 2D files (`SemiDegenerate.lean`,  $L = (\lambda/2)y^2 + x^4/24$ , 514 lines; `PureQuartic.lean`,  $L = x^4/24 + y^4/24$ , 478 lines) re-derived these as case-by-case proofs. The cleanup Tide refactors both files to apply the abstraction’s lemmas through a small `[simp]` bridge lemma, leaving the public theorem statements intact.

The Tide chains on top of `tide/separable-potential` per the linear-chain branch-hygiene rule; the new branch is `tide/twod-cleanup`, off Tide 12’s tip 725c76b. The Tide ran in a dedicated `git worktree` at `lean/laplace-cleanup/` — the first concrete application of the worktree-isolation fix that three prior retrospectives have recommended in response to the recurring concurrent-agent branch race.

## 2 The thing formalised

Three pieces, in order:

1. **AddSeparable.lean API tightening.** The `partitionFunction_addSeparable_factor` and `integral_separable_addSeparable` lemmas drop their integrability hypotheses, becoming unconditional (matching Mathlib’s `integral_prod_mul`, which is itself unconditional). The `gibbsExpectation_separable_addSeparable` and `gibbsCov_addSeparablefst_snd_eq_zero` lemmas drop the weight integrability hypotheses but keep the partition-function nonzero hypotheses (those are load-bearing — needed to divide).

**2. SemiDegenerate.lean refactor.** A new `[simp]`-tagged bridge lemma:

```
lemma semiDegeneratePotential_eq_addSeparable (lam : ℝ) :
  semiDegeneratePotential lam =
    addSeparable Laplace.OneD.quarticPotential
      (Laplace.OneD.harmonicPotential lam)
```

Then `partitionFunction_factor` and `integral_pow_pow_factor` become two-line proofs each:

```
theorem partitionFunction_factor (lam t : ℝ) : ... := by
  rw [semiDegeneratePotential_eq_addSeparable]
  exact partitionFunction_addSeparable_factor _ _ _
```

```
theorem integral_pow_pow_factor (lam t : ℝ) (m n : ℕ) : ... := by
  simp only [semiDegeneratePotential_eq_addSeparable]
  exact integral_separable_addSeparable _ _ _ (fun x => x ^ m) (fun y => y ^ n)
```

The `exp_neg_t.semiDegenerate_eq_mul` lemma becomes a similar two-line corollary of `exp_neg_t.addSeparable_eq_mul`.

**3. PureQuartic.lean refactor.** Mirror of step 2 with `pureQuarticPotential_eq_addSeparable` as the bridge.

**Public statements unchanged.** The `cov_affine_*` theorems and the moment lemmas (`gibbsExpectation_fst/snd/fst_mul.snd/fst_sq/snd_sq`) are unchanged in statement and continue to typecheck through the refactored proofs. Net delta: -22 lines across the three files (64 insertions, 86 deletions).

### 3 Proof strategy

The cleanup pivots on two simple observations:

1. **The bridge lemma is `[simp]`-tagged.** The semi-degenerate potential and the corresponding `addSeparable` are pointwise equal as functions  $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ . Tagging the equation `[simp]` lets a single `simp only` rewrite the integrand of `integral_pow_pow_factor` from the named potential to the `addSeparable` form, after which the `AddSeparable` lemma applies directly.
2. **Beta-reduction handles the lambda shapes.** `integral_separable_addSeparable U V t f g` expects  $f, g : \mathbb{R} \rightarrow \mathbb{R}$ . To recover the existing `integral_pow_pow_factor` statement, we apply at  $f = (x \mapsto x^m)$ ,  $g = (y \mapsto y^n)$ . Lean beta-reduces `(fun x => x ^ m)z.lto`

The API regression on `AddSeparable.lean` (dropping the integrability hypotheses) is a separate move: the underlying proof bodies were already unconditional via Mathlib's `integral_prod_mul`; the hypotheses had been added in Tide 12 as documentation discipline, not load-bearing. The cleanup Tide's concrete downstream demonstrates that the hypothesis-bearing form *blocks* the bridge-rewrite, since the existing `partitionFunction_factor` statements have no positivity hypotheses to source the integrability witnesses from. The simplest fix is to drop the discipline.

## 4 Roadblocks and resolutions

**The hypothesis package on AddSeparable blocks the refactor.** Tide 12 added Integrable hypotheses to `partitionFunction.addSeparable_factor` and friends, on GPT-5.5 Pro’s recommendation:

**GPT-5.5 Pro:** For Bochner  $\int$ , unconditional factorisation statements are brittle because the theorem you will actually use is typically the integrable version. ... Downstream files will naturally be able to apply [the hypothesis-bearing form], since for concrete potentials they already prove finiteness/integrability of the partition function.

The cleanup Tide is exactly that downstream, and the prediction did not hold. The existing `partitionFunction_factor` in both 2D files is *unconditional* (no positivity hypotheses, just  $\lambda$  and  $t$  as plain reals), so the wrapper has no integrability witnesses to supply. The choice was either to strengthen the wrapper statements with positivity (a public API change in two files), or to drop the hypotheses on the abstraction’s lemmas (a public API regression in one file). Both Claude and GPT (in the deliberation for this Tide) agreed that dropping the abstraction’s hypotheses is the smaller change; the discipline argument applies in principle, but in practice the unconditional form is what callers want.

This is worth recording as a Tide-skill calibration: “Bochner discipline” arguments are theoretically right but pragmatically lose to “match the existing caller’s signature shape”, at least until Mathlib itself decides to tighten its unconditional integral-prod lemmas.

**Worktree isolation: structural fix lands and works.** This Tide is the first to run in a dedicated `git worktree`. `lean/laplace-cleanup/` was created from `tide/separable-potential`’s tip and held its own `HEAD/index/working-tree` throughout the run. The parallel sessions in `lean/laplace/` (which had branched onto `tide/bounded-prior-continuous-test` and back to `main` several times during this session) did not interfere. Three previous retrospectives recommended this fix; this is the first concrete confirmation. Worth lifting to a Tide-skill preflight step.

## 5 What was Mathlib, what was new

**Mathlib provided.** Nothing new. The cleanup adds no new Mathlib API consumption.

**The seabed (Tide 12) provided.** The five generic factorisation lemmas of the `AddSeparable` module — the definition, the Boltzmann decomposition, partition factorisation, integral factorisation, expectation factorisation, and the mixed-covariance vanishing.

**What was new in this Tide.** The two `[simp]` bridge lemmas<sup>1</sup> identifying the existing 2D potentials as instances of `addSeparable`. Everything else is either deletion or a syntactic re-targeting of existing proof bodies through the bridge.

---

<sup>1</sup>Names: `semiDegeneratePotential.eq.addSeparable` and `pureQuarticPotential.eq.addSeparable`.

## 6 Lessons

- *Worktree isolation is the right structural fix.* Three prior retrospectives recommended this; one Tide actually used it; it worked. Promote to a Tide-skill preflight (“run the Tide in a `git worktree` when concurrent sessions exist on the seabed repo”).
- *The Bochner-discipline argument loses on first contact with downstream.* Tide 12 added `Integrable` hypotheses to factorisation lemmas as documentation discipline. Tide 13 (this one) removed them, because they blocked the cleanest refactor of two unconditional callers. The right convention for Mathlib-shaped factorisation lemmas with unconditional underlying API (`integral_prod_mul`) is: stay unconditional unless a specific caller’s needs strengthen the case. “Discipline” alone is not enough; concrete usage is.
- *Cleanup Tides have higher leverage than they look.* –22 lines is small. But the cleanup makes A2 (quartic–sextic mixed) and A4 (mixed even-power 2D moments) one-line specialisations rather than file-sized re-derivations. The next Tide that adds a 2D potential pays a constant cost, not a file cost.

## 7 Follow-ups

- *Quartic–sextic mixed potential (A2 from the 6 May survey).* Now a clean specialisation: define `quarticSexticPotential = addSeparable quarticPotential sexticPotential`, supply a one-line bridge, and the partition + moment factorisations come for free.
- *Mixed even-power 2D moments (A4).* One-line corollary of the integral-factorisation lemma at  $f = x \mapsto x^m, g = y \mapsto y^n$ , in either of the two refactored files (or generically over `addSeparable`).
- *Promote 2D atom integrability to `AddSeparable.lean`.* Both files still carry private `<name>_integrable_pow_pow` witnesses built via `Integrable.mul_prod` from two 1D witnesses. A generic `integrable_separable_addSeparable` in `AddSeparable.lean` would shrink each downstream file by another ~30 lines. GPT flagged this as a strong optional add-on; deferred to a follow-up Tide.
- *Generic `gibbsCov` API (orthogonal future Tide).* Still the natural infrastructure Tide for the affine-covariance Candidate B from Tide 12’s deliberation<sup>2</sup>. Independent of this cleanup but unblocked by it.
- *Promote worktree isolation to the Tide skill’s preflight.* The skill currently flags the concurrent-agent race as a recurring roadblock but does not prescribe a fix. Three retrospectives have named the same fix; one Tide has now shown it works. Worth landing.

## Acknowledgements

GPT-5.5 Pro’s deliberation contributed both the recommendation to keep public theorem statements unchanged (rejecting Candidate B’s definitional refactor) and the architectural decision to drop the integrability hypotheses on the `AddSeparable` lemmas rather than strengthen the wrapper

---

<sup>2</sup>Tentative names: `gibbsCov_symm`, `gibbsCov_add_left`, `gibbsCov_const_left`, `gibbsCov_smul_left` and right-hand variants.

statements with positivity. The full consult is preserved in the primer project's tide-log directory under the SRI repo.